

Caso 1: Fundamentos de POO

En este caso se desarrolló una clase Estudiante con atributos privados (nombre, edad y carrera), además de un constructor, métodos get/set y un método para mostrar la información. También se registraron tres estudiantes para comprobar el funcionamiento de la clase.

Análisis:

El encapsulamiento es uno de los principios fundamentales de la Programación Orientada a Objetos. Consiste en proteger los datos de una clase evitando el acceso directo a sus atributos mediante el uso de modificadores de acceso privados. Para interactuar con estos datos se utilizan métodos públicos conocidos como getters y setters.

El uso de constructores permite inicializar los objetos de manera ordenada y segura desde el momento de su creación. Gracias a ello se garantiza que cada estudiante tenga sus datos correctamente asignados, mejorando la organización y legibilidad del código.

Este enfoque favorece la seguridad, el mantenimiento y la reutilización de los programas.

Caso 2: Métodos, Static y Excepciones

En este caso se implementó una clase CuentaBancaria con un saldo privado y métodos para depositar y retirar dinero. Además, se utilizó un atributo static para llevar un contador de cuentas creadas y se aplicó el manejo de excepciones para validar operaciones incorrectas.

Análisis:

La palabra clave static permite que un atributo o método pertenezca a la clase y no a un objeto específico. Esto resulta útil cuando se necesita compartir información entre todas las instancias, como ocurre con el contador de cuentas.

El manejo de excepciones permite detectar y controlar errores durante la ejecución del programa. Por ejemplo, se puede impedir que un usuario retire una cantidad superior al saldo disponible o que ingrese montos negativos. Gracias a ello el sistema es más robusto y confiable.

La combinación de validaciones y excepciones mejora significativamente la experiencia del usuario y evita resultados inesperados.

Caso 3: UML y Análisis

En este caso se identificaron requerimientos funcionales y no funcionales, se elaboraron historias de usuario y se describió un diagrama de clases para representar la estructura del sistema.

Análisis:

El análisis previo es una etapa fundamental dentro del desarrollo de software. Permite comprender las necesidades del usuario antes de iniciar la programación, reduciendo errores y retrabajos.

Los requerimientos funcionales describen las acciones que debe realizar el sistema, mientras que los requerimientos no funcionales establecen características de calidad como rendimiento, seguridad o facilidad de uso.

Las historias de usuario ayudan a entender las necesidades desde la perspectiva del usuario final, mientras que los diagramas UML facilitan la representación visual de clases, atributos y relaciones. Todo ello contribuye a una mejor planificación y organización del proyecto.

Caso 4: Herencia y Polimorfismo

En este caso se creó una clase base Persona y dos clases derivadas: Estudiante y Docente. Se aplicaron los conceptos de herencia y polimorfismo para reutilizar atributos y comportamientos comunes.

Análisis:

La herencia permite que las clases hijas compartan características de una clase padre, evitando la duplicación de código. Gracias a ello se mejora la organización y se facilita el mantenimiento de los programas.

El polimorfismo permite que distintos objetos respondan de manera diferente a una misma operación. Esto proporciona flexibilidad al sistema y facilita futuras ampliaciones sin modificar la estructura principal.

La reutilización es una de las principales ventajas de estos conceptos, ya que reduce el tiempo de desarrollo y mejora la calidad del software.

Conclusiones

- La Programación Orientada a Objetos permite desarrollar aplicaciones más organizadas y fáciles de mantener.
- El encapsulamiento protege la información y mejora la seguridad de los datos.
- El uso de métodos static y excepciones contribuye a la creación de programas más robustos y confiables.
- El análisis previo mediante UML y requerimientos ayuda a reducir errores durante el desarrollo.
- La herencia y el polimorfismo favorecen la reutilización del código y facilitan la escalabilidad de los sistemas.